SMART INDIA HACKATHON 2026 · PROBLEM STATEMENT 26156 · NTRO

ULPF
The Complete Guide

Everything about this project, explained from zero — the problem, the market, the technology, the code, and every question a judge is likely to ask.

Universal Log Pre-processing Framework

Theme: Blockchain & Cybersecurity · Category: Software

Written for readers with no programming background.

34 SOURCE PACKS	**838,779** REAL RECORDS TESTED	**99.92%** COVERAGE
10,000 EVENTS / SECOND	**7** RUST CRATES	

What is in this guide

1 The problem, in plain English

What a log is · why this is hard · who is asking and why it changes everything

2 What we actually built

The five stages · the journey of one log line · what comes out

3 How the world solves this today

Market research · the seven commercial platforms · what every one of them misses

4 The technology, and why we chose it

Rust · OCSF · YAML parsers · no database · the cryptography

5 Just enough Rust to defend the project

Ownership · borrowing · Result · traits · reading our real code

6 Concepts you will be asked about

Hashing · hash chains · Merkle trees · signatures · how a web request works · the algorithms we use

7 Every feature: what, why, how

8 Problems we hit, and how we found them

The real war stories — the most useful section for the jury round

9 The numbers, and what each one means

10 Judge questions and our answers

31 questions researched from SIH juries and hackathon panels

11 Our honest weaknesses

12 One-page cheat sheet

HOW TO READ THIS

Read it three times, not once. The first pass gives you the story. The second pass makes the technology stick. The third pass is Part 10 alone — the questions — read out loud, answering before you look.

Nothing here needs another source. Every number in it was measured on this project, and every claim about other products came from research done specifically for this document.

PART 1

The problem, in plain English

Before any technology: what is actually broken in the world, and why does anybody care enough to run a national hackathon about it?

1.1 What is a log?

Every machine on a network writes a diary. Each time something happens — a connection allowed, a password rejected, a website fetched — the machine writes one line describing it. That line is a **log record**. A day's diary is a **log file**.

Here is one real line from a firewall in our test data:

```
Feb  1 00:00:02 bridge kernel: INBOUND TCP: IN=br0 PHYSIN=eth0 OUT=br0
SRC=192.150.249.87 DST=11.11.11.84 LEN=40 PROTO=TCP SPT=220 DPT=6129 SYN
```

Translated: on 1 February at 2 seconds past midnight, a machine at address `192.150.249.87` tried to open a connection to `11.11.11.84` on port 6129. Port 6129 was used by a well-known worm. This single line is potential evidence of an attack.

1.2 The scale of it

One firewall writes millions of these a day. A bank or a power grid operator runs thousands of devices. The problem statement asks for a system that can handle **billions of events per day**. To make that concrete:

Target	Meaning
1 billion events/day	11,574 events *every second*, without stopping
One event	roughly 150 characters
One day of them	about 150 gigabytes of pure text

1.3 Problem A — every device speaks a different language

There is no standard. Each manufacturer invented their own way of writing the same idea. Here are three devices describing *the same kind of event* — a connection:

```
Fortinet firewall:
  srcip=10.2.4.7 srcport=40589 dstip=8.8.8.8 action=accept

Squid web proxy:
  1756636800.123 152 10.0.0.50 TCP_MISS/200 12345 GET http://example.com

Suricata intrusion detector:
  {"src_ip":"10.0.0.5","dest_port":443,"event_type":"alert"}
```

Same meaning. Three completely different shapes. A security analyst who wants to ask "show me everything from address 10.2.4.7" has to know all three formats — and the other forty formats in the building.

The usual fix is to write a small program (a **parser**) for each device. That is slow, it must be written by a programmer, and it breaks whenever the manufacturer changes the format in a firmware update. The problem statement names this cost directly: *"Security teams often spend substantial effort developing source-specific parsers."*

1.4 Problem B — normal tools throw the evidence away

This is the deeper problem, and the one most people miss.

A normal log tool reads a line, pulls out the fields somebody thought of in advance, and stores those. Everything else is discarded. It looks harmless — until it matters.

> **WHY THAT IS DANGEROUS**
>
> Six months later there is an investigation. The tidy record in the dashboard says "connection from 10.2.4.7 was blocked." A lawyer asks: *how do you know the device actually said that, and that nobody edited it since?*
>
> With a normal tool there is no answer. The original line is gone. All that remains is somebody's summary of it, sitting in a database that any administrator could have changed.

Two separate failures are hiding in there:

- **Loss.** The original is gone, so anything the parser did not extract is gone forever — including the part that turns out to matter.
- **No proof.** Even the part that was kept has nothing showing it is unaltered. A stored record and a tampered record look identical.

1.5 Who is asking — and why it changes everything

The problem statement comes from **NTRO**, India's technical intelligence agency. The contact address on it belongs to **NCIIPC** — the National Critical Information Infrastructure Protection Centre, which protects power grids, banking, telecom, transport and defence.

That single fact explains three requirements that would otherwise look like over-engineering:

Requirement	Why this customer demands it
Air-gapped (no internet, at all)	Critical networks are physically disconnected by policy. Software that phones home, downloads a font, or calls a cloud AI simply cannot be installed.
Forensic preservation	Logs from a power grid become evidence — in an investigation, and possibly in court. "This is what the device said" has to be provable, not asserted.
Massive scale	National infrastructure produces billions of events daily.

THE SENTENCE THAT DEFINES OUR SCOPE

The statement's Background lists every kind of computer. But its **Current Scope** narrows it: *"converts any **perimeter network device**-generated log or event — regardless of source, format, vendor, or technology."*

Perimeter devices are the machines at the edge of a network, where it meets the outside world: firewalls, intrusion detectors, proxies, VPNs, web servers. That is what we built for, and it is why our headline number counts only those.

PART 2

What we actually built

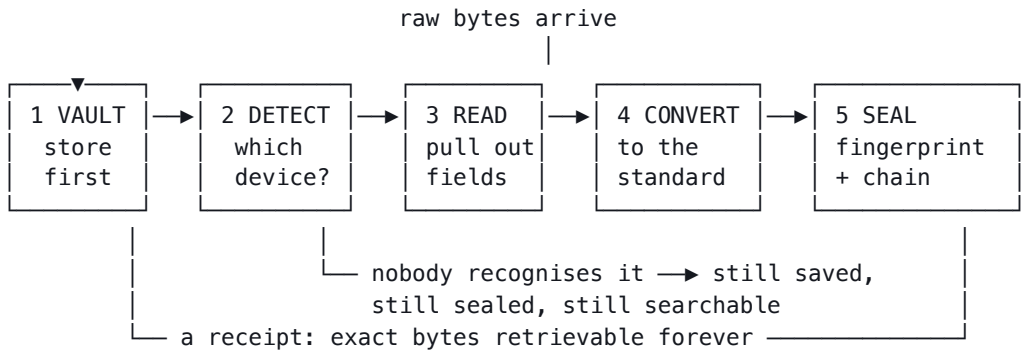
One program. Logs go in, one standard format comes out, and nothing is lost or unprovable on the way.

2.1 The one-paragraph version

> IF YOU REMEMBER ONE THING
>
> **ULPF takes logs from any security device, converts them all into one standard format, and keeps mathematical proof that nothing was altered.** It stores the original bytes *before* it tries to understand them, so nothing is ever lost. Adding support for a new device means writing a small text file, not a program. And it runs on a machine with no internet connection.

2.2 The journey of one log line

Every record goes through five stages, always in this order. **The order is the design** — it is the single most important thing to understand about the project.



Stage 1 — Vault (store it before understanding it)

The exact bytes are written to a compressed, **append-only** file. Append-only means you can add to the end but never edit what is already there — like a bank ledger written in pen.

You get back a **locator**, which is a receipt:

```
ulpf:raw:0000000000000000:0000000000000000:0000008b
          ↑ which file      ↑ position        ↑ length
```

Every record produced later carries this receipt, so the original can be fetched back in one step at any time.

> **WHY STORE FIRST, BEFORE PARSING?**
>
> Because everything after this point *can fail*. An unknown device, a corrupted line, a bug in our own code — none of them can lose the original, because the original was already safe before we touched it.
>
> Most tools do this backwards: they parse first and store what they understood. We store first and understand second. That one reversal is what makes "nothing is lost" true rather than aspirational.

Stage 2 — Detect (which device is this?)

We check the text against a list of recognisers. A Fortinet firewall always writes `devname=`; a Squid proxy always writes `TCP_`. The first recogniser that matches wins. This is a simple text search, so it is very fast.

Stage 3 — Read (pull the fields out)

Ten small readers do the actual work. The insight that keeps the number at ten: devices do not each invent a format — they wrap one of about six *bodies* inside one of about two *envelopes*. So we combine readers in a chain instead of writing one parser per device.

Device	Chain used
Fortinet firewall	syslog → key=value
Palo Alto firewall	syslog → csv
Suricata detector	json
Snort detector	syslog → regex

Same ten readers, different orders. That is why device number 35 needs no new code.

Stage 4 — Convert (into one standard shape)

The fields are mapped onto **OCSF**, an open industry standard. Fortinet's `srcip` and Palo Alto's `col.7` both become `src_endpoint.ip`. Anything we extracted but had no home for is kept in a section called `unmapped` rather than thrown away.

Stage 5 — Seal (make tampering detectable)

The finished record is fingerprinted, linked to the record before it, and added to a structure that allows proving a single record later. Part 6 explains exactly how.

2.3 And if nobody recognises the device?

This is the case the problem statement cares most about, and where most tools give up. Three things still happen:

1. **It is still saved** — the vault already has it.

2. **It is still sealed** — it gets a fingerprint and joins the chain.
3. **It is still searchable.** Our **salvage** feature reads *any* text and pulls out addresses, ports, web links, email addresses and machine names — with no parser at all.

Here is a completely invented device format that no parser knows, and what ULPF recovered from it anyway:

```
Input (a format we made up on the spot):
ZORPFW-9000 flow-end 10.2.4.7:44321 -> 93.184.216.34:443
            verdict=allow user=jdoe@corp.example.com

Recovered with no parser:
  IP address   10.2.4.7
  IP address   93.184.216.34
  Port         44321
  Port         443
  Email        jdoe@corp.example.com
```

An analyst hunting that address will now find this record. Before we built salvage, this record was a meaningless blob of text.

PART 3

How the world solves this today

Researched specifically for this document. If a judge asks "does this already exist?", this is the honest answer — including where we are behind.

3.1 Yes, there is an industry for this

The category is called **Security Data Pipeline Platforms** (SDPP). It exists for a commercial reason: most SIEMs (the big security dashboards) charge *per gigabyte ingested*. Log volume grows faster than budgets, so companies now put a filtering layer in front to cut what reaches the expensive system.

Note what that means: **the industry was built to reduce cost, not to preserve evidence.** That difference explains almost everything in this section.

3.2 The main players

Product	What it is	Known weakness
Cribl	The category leader. Founded 2018, roughly half the Fortune 100 as customers, crossed $300M annual revenue in early 2026.	Its storage layer is **cloud-only**, at about double raw S3 cost.
DataBahn	Founded 2023. Markets an "Agentic Data Control Plane" — heavy use of AI for mapping.	No storage layer at all.
Axoflow	Founded by the creator of syslog-ng. The one competitor that runs **on-premises and air-gapped**.	Smallest of the group; no integrity story.
Monad	Routes to external platforms.	Poor fit for on-premises deployments.
VirtualMetric	Specialised for Microsoft Sentinel.	Narrow outside the Microsoft world.
Falcon Onum	Acquired by CrowdStrike.	No longer independent.
Vector, Fluent Bit, Logstash	Free, open-source, very mature, and **faster than us**.	None of them does integrity. None fingerprints or chains a record.

3.3 The finding that matters most

The most thorough market comparison published in 2026 is Axoflow's own review of the seven platforms. It argues the other reviews miss two decisive criteria:

1. Whether the platform can keep data **on-premises**;

2. Whether normalization is **deterministic or probabilistic** — fixed rules versus AI guessing.

> **THE GAP IN THE WHOLE CATEGORY**
>
> That review — the most detailed analysis of this market that exists — **does not mention data integrity, tamper-evidence, or chain of custody once.** The category does not even have a column for it.
>
> That is not because it is unimportant. It is because these products were built to reduce SIEM bills, and nobody buying them was asking "can you prove this record is unaltered?" NCIIPC is asking exactly that.

3.4 On the AI question — we agree with the critique

Several competitors now put AI in the live data path to guess field mappings. The published critique is blunt, and it is worth memorising because a judge may ask why we did not use more AI:

"AI autonomy is appropriate for tasks where errors are recoverable. Normalization is not one of them."

If an AI guesses wrong about which field is the source address, every detection rule built on that field silently breaks. Our design takes the opposite position: **a model may help write a parser, but no model ever runs while records are being processed.** Once a human approves a parser, its behaviour is fixed and reproducible forever.

3.5 So where are we genuinely different?

Capability	Them	Us
Normalize many formats to one schema	Yes — mature	Yes
Runs fully air-gapped	Only Axoflow	Yes, proved in CI
Stores original bytes before parsing	No	**Yes**
Every record fingerprinted and chained	No	**Yes**
Prove one record without revealing the rest	No	**Yes**
Raw throughput	**Faster**	10,000/sec per collector
Number of supported devices	**Hundreds**	34
Maturity, support, ecosystem	**Years ahead**	A hackathon project

SAY THIS HONESTLY IF ASKED

The individual pieces all exist somewhere. Hash chains are old. Merkle trees come from Certificate Transparency. OCSF is an open standard we did not invent. Two products — AuditKit and Caver — do tamper-evident evidence handling, though neither is a log normalization framework.

The combination is what we did not find anywhere: store-before-parse, plus parsers as data files, plus a vendor-neutral schema, plus per-record proofs, in one air-gapped binary. Claiming the parts are novel would be false. Claiming the combination is rare is true, and it is a stronger position because it survives scrutiny.

PART 4

The technology, and why we chose it

Judges reliably ask "why this language / why this database". The winning answer names the *constraint* that forced the choice — never popularity.

4.1 Why Rust?

Rust is a programming language built for software that must be both fast and safe. Three constraints in this project pointed at it:

Constraint	Why Rust answers it
Speed — 10,000+ records/second	Rust compiles to machine code, like C. There is no interpreter and no pause to clean up memory, so speed is steady rather than jumpy.
Safety — we read data attackers control	Log records arrive from the network. In C, a mistake reading attacker-controlled bytes becomes a security hole. Rust makes whole classes of that impossible at compile time.
One file, no installer	Rust produces a single executable with nothing to install alongside it. On an air-gapped machine that matters enormously.

Why not the obvious alternatives

Language	Why not
Python	Roughly 10–100× slower here, and needs an interpreter plus libraries installed on the target machine.
Java	Fast enough, but needs a Java runtime installed, and its automatic memory cleanup causes pauses — bad when records arrive continuously.
Go	Genuinely a reasonable choice — single binary, fast. We chose Rust for stricter safety guarantees and no cleanup pauses. This is the one comparison where the honest answer is "Go would also have worked."
C / C++	Fast, but a memory mistake while reading hostile input is a vulnerability. Not acceptable for a security tool.

4.2 Why OCSF as the standard format?

OCSF — Open Cybersecurity Schema Framework — is an open standard, originally from AWS, that defines what a security event looks like: which fields exist and what the numbers mean.

The alternatives are all controlled by one vendor:

Standard	Owner	Problem
OCSF	Open / vendor-neutral	*— chosen*
ECS	Elastic	Tied to one company's products
CIM	Splunk	Tied to Splunk
ASIM	Microsoft	Tied to Sentinel
UDM	Google	Tied to Chronicle

> **THE SECOND REASON, AND THE BETTER ONE**
>
> OCSF defines a `record_integrity` profile — an official, standard place to put fingerprints and chain links. Our entire integrity design hangs on that hook. We are not bolting on a private invention; we are using a part of the standard that already exists, which means anyone else's OCSF tool can read our records.
>
> We ship a copy of the standard inside the repository, unmodified, at a named version. Any reviewer can download the original and compare — proving we did not quietly edit a definition to make our output look correct.

4.3 Why are parsers text files instead of code?

A parser in ULPF is a **Source Pack** — one YAML file. YAML is a plain-text format designed to be readable by people. Here is a real, complete section from ours:

```yaml
identity:
  id: openssh-auth
  detect:
    - contains_all: ["sshd["]        # how to recognise it

extract:
  - decoder: syslog                  # which readers, in order
  - decoder: regex

map:
  class_uid: 3002                    # it is an Authentication event
  src_endpoint.ip:
    from: src                        # their field  →  the standard field
    observable: ip
```

Four consequences, and each answers a different requirement:

- **No programmer needed.** A security analyst can add a device.
- **No rebuild, no restart.** A file-watcher notices the new file and loads it within a second, while the system is running.

- **It can be reviewed.** A human reads the recognition rules and the mapping before approving it — impossible with compiled code.
- **It carries its own tests.** Each pack embeds real sample lines and the values they must produce.

4.4 Why no database?

A normal system of this kind would store records in PostgreSQL or Elasticsearch. We deliberately use plain files. Three reasons:

- **Air-gap.** A database is another thing to install, license, patch and back up on an isolated machine.
- **Append-only is the point.** A database is designed to let you `UPDATE` rows. For evidence, being able to edit is the opposite of what we want.
- **Speed.** Appending to a file is about as fast as a computer can write.

Instead we wrote a purpose-built store: records grouped into roughly 1 MB blocks, compressed with zstd, each block carrying a checksum, and an index so any single record can be fetched in one disk seek.

4.5 The cryptography, and why each piece

Tool	Job	Why this one
SHA-256	Fingerprints a record	The only fast option OCSF names explicitly, so any third party can verify our records with standard software and no special knowledge.
BLAKE3	Optional faster fingerprint	Faster, but not in the OCSF list, so it must declare itself as "Other" — a real trade-off we made visible rather than hiding.
Ed25519	Signs checkpoints	Modern, small, fast signatures. Signing is what turns "you can detect tampering" into "you cannot forge a replacement."
Merkle tree (RFC 6962)	Prove one record	The exact construction used by Certificate Transparency, which secures HTTPS worldwide. We did not invent a scheme; we used the proven one.
RFC 8785 (JCS)	Makes fingerprints reproducible	Guarantees two different programs turn the same record into the same exact bytes, so they compute the same fingerprint.
zstd	Compression	Fast, and log data repeats heavily — we measure better than 4:1 on real firewall logs.

PART 5

Just enough Rust to defend the project

You do not need to write Rust. You need to read a screen of it and explain what it does. This part gives you exactly that, using our real code.

5.1 The one big idea: ownership

Every other language handles memory in one of two ways. Rust invented a third.

Approach	Used by	Cost
Programmer frees memory manually	C, C++	Forget once → crash or security hole
Automatic cleaner runs in background	Java, Python, Go	Safe, but pauses and slower
Compiler proves it at build time	**Rust**	**Safe and fast — but the compiler refuses bad code**

The rule: **every value has exactly one owner, and when the owner finishes, the value is freed.** The compiler checks this before the program ever runs.

> SAY IT LIKE THIS
>
> "Rust checks memory correctness while compiling, so bugs that would be crashes or vulnerabilities in C become compile errors we have to fix before the program can even be built. For software that reads attacker-controlled input, that is worth the strictness."

5.2 Borrowing

If a value has one owner, how do two parts of the program use it? They **borrow** it — marked with `&`.

```
fn decode(&self, input: &str) -> Result<Decoded>
```

`&str` means "I am *borrowing* some text; I will read it, not own it, and I will not keep it after I return." This is why ULPF is fast: reading a field does not copy the text, it just points into the original line.

5.3 No exceptions — `Result` and `Option`

Rust has no exceptions. A function that can fail says so in its type, and the caller *must* handle it.

```
Result<T, E>   →  either Ok(value) or Err(problem)
Option<T>      →  either Some(value) or None
```

You cannot forget to check. The compiler will not build the program.

> **WHY THIS MATTERS FOR US SPECIFICALLY**
>
> Our release build is configured to **abort on panic** — any unexpected crash kills the whole collector. So every decoder is written to return `Err` rather than crash, and we have a test that throws deliberately hostile input at all ten of them to prove it. This is not theoretical: a real crash bug hid in our XML reader until somebody fed it a Japanese log line.

5.4 Traits — the plug socket

A **trait** is a contract: "anything that implements this must provide these functions." Ours:

```
pub trait Decoder {
    fn name(&self) -> &'static str;
    fn decode<'a>(&self, input: &'a str) -> Result<Decoded<'a>>;
}
```

All ten readers implement this. The rest of the program does not care which one it holds — it just calls `decode`. That is how a Source Pack can name any chain of readers and the pipeline works without changes.

5.5 Crates and Cargo

A **crate** is a Rust package. **Cargo** is the build tool. Our project is seven crates, each with one job:

Crate	Its one job
`ulpf-core`	The shared vocabulary — the basic types everything passes around
`ulpf-vault`	The evidence store — append-only, compressed, retrievable
`ulpf-decode`	The ten readers, plus salvage extraction
`ulpf-pack`	Source Packs — the file format and how to run one
`ulpf-ocsf`	The standard output shape, and all the cryptography
`ulpf-generator`	Grouping unknown logs and drafting new parsers
`ulpf-cli`	The actual program: commands, web console, outputs

Splitting like this means the cryptography crate cannot accidentally depend on the web console, and each part can be tested alone.

5.6 Reading real code from our project

This is the heart of the whole system — the function that processes one record. If you can explain this screen, you can defend the architecture.

```
pub fn process(&mut self, raw: &[u8], envelope: &Envelope)
    -> anyhow::Result<Processed> {

    // Stage 1: preserve. Nothing below this line can lose the original.
    let raw_ref = self.vault.append(raw)?;

    let text = String::from_utf8_lossy(raw);

    // Stage 2 + 3 + 4
    let (mut event, disposition) = match active_packs.identify(&text) {
        None => (self.minimal_event(raw, envelope, &event_uid)?,
                 Disposition::Unidentified),
        Some(pack) => match pack.extract(&text) { ... }
    };

    // Stage 5: attest, linking this event to its predecessor.
    self.attestor.attest(&mut event)?;

    Ok(Processed { event, disposition, raw_ref })
}
```

`&mut self`	this function changes the pipeline's own state (the vault grows, the chain advances)
`raw: &[u8]`	borrowed raw bytes — *not* text, because a log may contain invalid characters
`?`	"if this failed, stop and return the error" — the whole error-handling style in one character
`match`	handle every possible case; the compiler checks none is missed
`None =>`	no parser recognised it — and note it still produces an event rather than dropping it

THE LINE THAT PROVES THE DESIGN

`let raw_ref = self.vault.append(raw)?;` comes **before** any attempt to understand the record. If a judge asks "how do you know nothing is lost?", point at this line. Everything after it is allowed to fail.

PART 6

Concepts you will be asked about

The cryptography, the web layer, and the algorithms — each explained from zero, with the version of the answer you can say out loud.

6.1 Hashing — the digital fingerprint

A **hash function** takes any amount of data and produces a fixed-length code. SHA-256 always produces 64 characters, whether you feed it one word or a whole book.

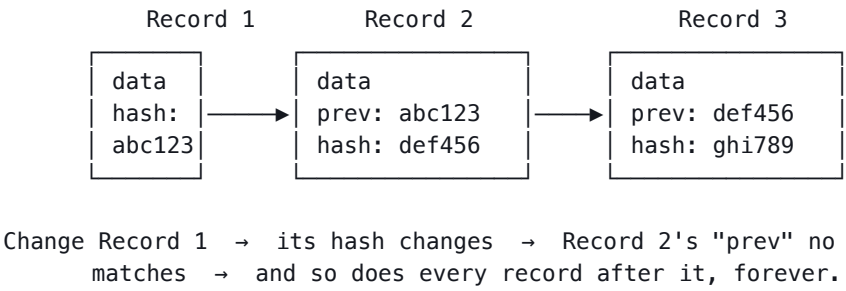
```
"hello"        →  2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e7304336
"hellp"        →  9c8f0f6b1c1e2b0d4a9f5e3c7d2a1b8e4f6c9d0a3b5e7f1c4d8a2b6e
      ↑ one letter different  →  a completely different fingerprint
```

Three properties make it useful as evidence:

- **Same input always gives the same output.** Recompute it later and compare.
- **Any change gives a wildly different output.** Even one character.
- **You cannot work backwards** from the fingerprint to the data.

6.2 Hash chain — making order tamper-evident

A fingerprint proves one record is unaltered. A **chain** proves the sequence is. Each record's fingerprint is computed over its own content **plus the fingerprint of the record before it**.



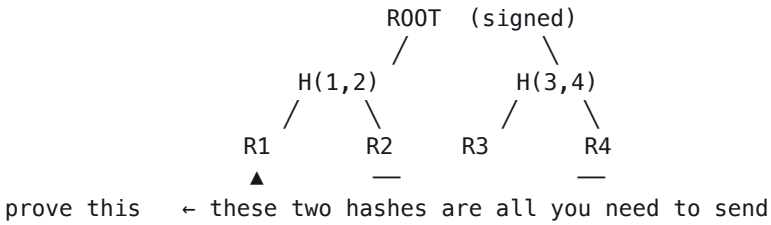
Say it like this: "Each record contains the fingerprint of the one before it, so altering any record breaks every record that follows. You cannot quietly edit the middle of the log."

6.3 Merkle tree — proving one record without showing the rest

This is our strongest technical feature, and the clearest answer to the Blockchain theme.

The problem it solves: a chain proves the whole log is intact, but only by replaying all of it. Suppose you need to prove *one* record out of a million — in court, to an auditor. Handing over all million records is slow, and for a classified log it is simply not permitted.

A Merkle tree pairs up fingerprints and hashes each pair, repeatedly, until one value remains at the top — the **root**.



To prove record 1 belongs: send record 1 plus the fingerprints of R2 and H(3,4). The verifier recomputes upward and checks the root matches the signed one. **They learn nothing about records 2, 3 or 4.**

Records in log	Hashes needed to prove one
1,000	10
1,000,000	20
1,000,000,000	30

Measured on our system: **11 hashes, 352 bytes.**

> **WHY THIS ANSWERS "BLOCKCHAIN & CYBERSECURITY"**
>
> A Merkle tree is the data structure underneath every blockchain. We use the version defined in RFC 6962, the standard behind Certificate Transparency, which protects HTTPS across the whole internet.
>
> We did **not** build a cryptocurrency or a distributed ledger — that would be absurd for a log collector. We took the part of the technology that genuinely applies: append-only records, hash chaining, signed commitments, and proofs of membership.

6.4 Digital signatures — from "detectable" to "unforgeable"

A hash chain has one weakness: an attacker who replaces the *entire* log can recompute all the fingerprints, and it will look perfectly consistent.

A **signature** closes that. There are two keys: a **private key** that only the collector holds and signs with, and a **public key** anyone can use to check. Without the private key you cannot produce a valid signature.

Why we sign periodically, not per record

Two reasons, and both are worth knowing:

- **The standard has nowhere to put it.** OCSF's signature object describes a signature but has no field to carry the bytes.
- **Cost.** We measured it: signing every record gave **102 records per second**. Signing every 500 records instead gives 10,000 — a 100× difference for the same guarantee, because the per-record chain already detects tampering.

6.5 How a web request works (our console)

The console is the web page an operator looks at. Here is the full round trip:

```
1  Browser: user opens http://127.0.0.1:8787
2  Browser sends an HTTP GET request to that address
3  Our program (the "server") receives it
4  Server checks: is this request allowed?  (see below)
5  Server sends back HTML  →  the page appears
6  The page's JavaScript asks for data: GET /api/stats
7  Server replies with JSON  →  numbers update on screen
```

HTTP is the language browsers speak. **GET** means "give me something"; **POST** means "here is something, do it." **JSON** is a text format for structured data. The console repeats step 6 on a timer, which is why the numbers update live.

The security check at step 4

Our console has no login. So we check two things on every request:

- **Origin** — browsers attach this to requests coming from another website, and a web page cannot fake it. This stops a malicious site you have open in another tab from silently installing a parser.
- **Host** — must be an address we actually listen on. This stops a trick called DNS rebinding.

Be honest about this one: it is browser safety, not authentication. Anyone who can reach the port can use it. That is why it listens only on the local machine by default.

6.6 The data structures and algorithms we actually use

Technique	Where	Why that one
Binary search *O(log n)*	Finding which block holds a record	Blocks are in order, so we halve the search each step. 1,000,000 blocks → 20 steps.
Array with linear scan *O(n)*	The extracted fields of one record	Deliberately *not* a hash map. A record has tens of fields, not thousands, and at that size scanning a contiguous list is faster than hashing.
Merkle tree *O(log n)*	Proving one record	Turns a million-record proof into twenty hashes.
Drain clustering	Grouping unknown logs	Buckets lines by word count, then compares position by position. Turns 4,812 unknown lines into one reviewable pattern.
Greedy farthest-point	Choosing example lines	Picks the *most different* examples rather than the first few. Taken from two research papers (LILAC, MicLog).
Sorted map (BTreeMap)	Fields inside a record	Order must be identical every time, or the fingerprint would change for identical data and verification would break.

PART 7

Every feature: what, why, how

Each feature exists because a specific requirement demanded it. This is the mapping.

Feature	Requirement	How it works
Raw vault	(a) preserve without loss	Append-only file, ~1 MB compressed blocks, checksum per block, index for one-seek retrieval. Written *before* parsing. Survives a power cut: a crash costs the last block, not the archive.
Ten decoders	(b) extract attributes	syslog (2 dialects), key=value, CSV, CEF, LEEF, JSON, XML, regex. Composed into chains by each pack.
OCSF mapping	(c) common taxonomy	Each pack maps its fields onto standard names. Unmapped fields are kept, not dropped.
Locator + fingerprint + chain + Merkle proofs	(d) traceability	Four layers: a receipt to fetch the original, a fingerprint proving it is unaltered, a link to the previous record, and a proof for any single record.
Source Packs + hot reload	(e) plug-and-play	YAML files in a folder. A watcher reloads within a second. A broken file is skipped and logged; the working set stays live.
Web console	(f) unified visibility	Live view, event inspector, unknown-log clusters, parser list, integrity page. Compiled into the binary — no internet needed.
Output sinks	(g) SIEM / data lake	NDJSON by default; Parquet, OpenSearch and Splunk as options, all batched.
Feature table	(h) AI/ML-ready	A fixed 24-column table for machine learning. The columns never change when a parser changes — so a model trained last month still reads this month's data.
Clustering + two generators	(i) reduce parser effort	Unknown logs are grouped; a candidate parser is drafted (with or without AI), scored against real samples, and activated only after a human approves.
Salvage extraction	(i) + Current Scope	Pulls addresses, ports, links and names out of *any* record, even one no parser claimed. Deterministic — no model.
Everything compiled in	(j) air-gapped	No CDN, no web font, no telemetry, no cloud AI. Proved in CI by building and testing with the network disabled.
Container	(k) platform independent	Two-stage build onto a minimal image with no shell, read-only filesystem, all privileges dropped. 65 MB.

7.1 The one feature worth demonstrating live

THE TAMPER TEST

On the Integrity page, one button alters a single field on one stored record — changing a source IP address. Verification immediately reports:

```
fingerprint mismatch: event content has been altered
```

Crucially the vault is *not* touched, which is the whole point: the original bytes remain retrievable and provably different from the altered version. This is a live test on real data, not a slide.

PART 8

Problems we hit, and how we found them

Judges ask "what was the hardest bug?" and "what did you get wrong?". These are real, and each one taught us something worth saying out loud.

8.1 The vendor manual was wrong

We wrote a parser for the Blue Coat proxy from the manufacturer's official documentation. It passed 100% of its own tests. Then we fed it 8.1 million real Blue Coat records.

It matched zero of them.

The real log file contains a header line listing its own field order — and it disagreed with the manual in two places. We rewrote the parser from the file's own header: **0% → 99.01%.**

> THE LESSON
>
> A test written by the same person who wrote the parser proves only that they agree with themselves. This is why we insist on real captured data, and why we list which parsers still lack it.

8.2 Tests passing while the system silently broke

We added a parser for the Thunderbird supercomputer logs. It recognised lines containing `(pam_unix)`. Every test passed.

But ordinary Linux logs contain `(pam_unix)` too. The new parser grabbed them first, then failed to read them. **Linux coverage fell from 94% to 19% — with a completely green test run.**

The fix was not just the parser. We added a permanent check to the test suite: for every sample line, confirm that *this* parser claims it and not a different one. It immediately found two more cases we did not know about.

8.3 A parser generator that produced useless parsers

Our system can draft a parser automatically from unknown logs. It reported "drafted 20 candidates" and looked like it worked.

Every one contained a **timestamp** in its recognition rule:

```
contains_all: ["20171223-22:15:29:606|Step_LSC|..."]
```

A timestamp happens once and never again, so each parser could match exactly one line — ever. It looked like progress and was worth nothing.

We also found the grouping split lines on spaces only, so formats separated by `|` never grouped: 2,000 records became clusters of four. After fixing both, the top 20 clusters covered **1,902 of 2,000 records**.

8.4 Full coverage that was actually blindness

We removed the firewall parser to see what would happen. The generic syslog parser claimed 100% of the lines and reported **full coverage**.

But it extracted almost nothing — a firewall log full of IP addresses had become unsearchable while the dashboard showed success. Salvage had not run, because the record was technically "parsed."

We changed salvage to run on *every* record, adding only what the parser missed. Result on that same data: **0% → 100% of records carrying searchable indicators.**

> **THE LESSON WORTH REPEATING**
>
> "A parser claiming a record is not the same as a parser understanding it." Our coverage number counts recognition. We now measure extraction separately, because the first number can look perfect while the system is useless.

8.5 A crash hiding in the XML reader

Our XML reader stepped through text one byte at a time. That works for English. It crashes on Japanese, where one character occupies several bytes — and the release build is set to abort on any crash, meaning the whole collector would die.

There was a fuzzing tool meant to catch this, but it required a special compiler and so had never actually run. We replaced it with a test that runs on every commit, feeding deliberately hostile input — multi-byte characters at every position, truncated records, nested structures — to all ten readers.

8.6 A false claim in our own README

Our README stated: *"no log line anywhere in this project is synthesised."*

It was not true. A sample file contained 7,319 invented lines. We found it by checking ourselves and corrected the claim to the one that is actually true — *no coverage or throughput figure comes from synthesised data* — and labelled the file clearly.

> **WORTH SAYING TO A JUDGE**
>
> "We audited our own claims and found one that was false, so we corrected it." That answers the honesty question better than never having had a problem.

PART 9

The numbers, and what each one means

Know these. Judges test whether you understand your own figures or merely memorised them.

9.1 Coverage

Source	Origin	Records	Coverage
iptables firewall	Honeynet	179,752	100.0000%
Snort detector	Honeynet	69,039	99.9986%
Dragon detector	Honeynet	42,899	100.0000%
Squid proxy	Honeynet	533,197	99.9771%
Apache web	Honeynet	3,554	99.9719%
Linux, SSH, mail, Proxifier	Honeynet + Loghub	10,338	81–100%
Total (perimeter)		**838,779**	**99.9219%**

A further twelve corpora outside the perimeter scope are measured separately: **862,779 records at 99.7175%** across all 23.

> THE NUMBER TO LEAD WITH
>
> The **iptables 100%** figure is genuine cross-validation. That parser was written against a *completely different* 307,524-record capture and never adjusted for this one. It is the strongest single fact we have, because it rules out the obvious criticism that we tuned the parser to the test.

9.2 Speed

Offered rate	Sent	Received	Loss
10,000 /sec	100,000	100,000	0%
12,000 /sec	120,000	118,000	1.7%
15,000 /sec	150,000	125,000	16.7%

10,000 per second is our honest sustained figure — 864 million a day. The billion-a-day target needs 11,574, so one collector reaches 86% of it.

We do not quote the 12,000 figure, because it drops 1.7% of records. For a log collector, that is not a rounding error — it is the exact failure the whole design exists to prevent.

How we get past it

Run more collectors. Each is independent — own port, own store, own key, own chain. Measured: two collectors, 120,000 events, **zero loss**, each chain verifying separately. Five collectors would reach 50,000/sec.

9.3 Everything else

Source Packs	34
Embedded tests, all passing	64
Decoders	10
Lines of Rust	~18,400
Container image size	65.4 MB
Proof size for one record	11 hashes / 352 bytes
Compression on real firewall logs	better than 4:1
Parsers with real-data proof	25 of 34

PART 10

Judge questions and our answers

Researched from SIH jury guidance and hackathon panels. The questions are real and recurring; the answers are specific to this project. Practise saying them aloud.

A. About the demo

Q1 Which part of what we just saw is real, and which is mocked?

TRAP: vagueness. Judges reward honesty and punish discovered mocks.

OUR ANSWER

All of it is real. The log data comes from public research archives — Honeynet and Loghub — and is used unmodified. The coverage figure is measured by a script anyone can re-run. One file in the project is synthetic and it is labelled as such: a sample file for trying the system without downloading 200 MB. It is never measured and appears in no published number.

Q2 Show me this running. Not the slides.

OUR ANSWER

Start the console, switch on two live sources, then go straight to the Integrity page: verify the chain, then press the tamper test. It alters one IP address on one record and verification immediately reports *"fingerprint mismatch: event content has been altered."* Sixty seconds, live, on real data.

Q3 Can you try it with this input?

OUR ANSWER

Yes — paste any log line into the Event inspector. If no parser recognises it, that is the interesting case: it is still stored, still fingerprinted, and salvage still pulls out the addresses and ports. We have demonstrated this with formats invented on the spot.

Q4 What happens when a dependency is down or slow?

TRAP: "It wouldn't be."

OUR ANSWER

We have almost none by design. There is no cloud service, no database, no internet. The one optional dependency is a local AI model: if it is unreachable, the system falls back automatically to the rules-based generator and reports the model as offline. If a SIEM we forward to is down, records are still stored and still sealed — forwarding is a fan-out, never the system of record.

B. About the technology

Q5 Why Rust? Why not Python or Java?

TRAP: "It's fast and modern." Name the constraint instead.

OUR ANSWER

Three constraints. We need 10,000+ records per second, so an interpreted language is out. We read attacker-controlled bytes from the network, and Rust makes memory bugs compile errors instead of vulnerabilities. And it produces one file with nothing to install, which matters enormously on an air-gapped machine. Go would also have been a reasonable choice; we preferred Rust's stricter guarantees and lack of collection pauses.

Q6 Why OCSF and not Elastic ECS or Splunk CIM?

OUR ANSWER

Every alternative is controlled by one vendor — ECS by Elastic, CIM by Splunk, ASIM by Microsoft, UDM by Google. Choosing any of them ties a national agency to one company. OCSF is genuinely vendor-neutral. It also defines a `record_integrity` profile, which is the standard hook our whole integrity design uses — so we are not inventing a private scheme.

Q7 Why no database?

OUR ANSWER

Three reasons. A database is another thing to install and patch on an isolated machine. Databases are built to let you update rows, and for evidence that is exactly what we do not want. And appending to a file is about as fast as a computer can write. We wrote a purpose-built append-only store instead: compressed blocks, checksums, one-seek retrieval.

Q8 Why not use AI to do the parsing directly?

OUR ANSWER

Because a wrong field mapping silently breaks every detection rule built on it. The published critique of this market puts it well: AI autonomy suits tasks where errors are recoverable, and normalization is not one of them. Our hot path is entirely deterministic — no model runs while records are processed. A model may *draft* a parser, but it is scored against real samples and approved by a human, and from that moment its behaviour is fixed.

Q9 What was the hardest bug, and how did you find it?

OUR ANSWER

A parser we added recognised lines containing `(pam_unix)`. Ordinary Linux logs contain that too, so it claimed them and then failed to read them — Linux coverage fell from 94% to 19% *with every test passing*. We found it because we re-measure against real data after every change. The fix was not just the parser: we added a permanent check that verifies each sample line is claimed by its own parser and not a different one. It immediately found two more cases.

Q10 How did you test the AI part?

OUR ANSWER

By holding a parser out and measuring whether the generator could rediscover it. We removed one, fed the system that device's logs, let it draft parsers automatically, and measured coverage: 0% → 47.6% with no human writing anything. We also measured extraction quality separately and found the drafted parsers were producing zero OCSF fields on some formats, which we then fixed.

Q11 Where does the data sit, and who can see it?

OUR ANSWER

Entirely on the operator's own machine — there is no cloud component. The console listens on the local machine only by default. Being honest: it has no login. It blocks requests from other websites and rebound addresses, but anyone who can reach the port can use it, so any wider exposure needs an authenticating proxy in front. That is documented in our security notes rather than hidden.

C. About the problem and users

Q12 How does the user do this today, without you?

OUR ANSWER

They write a parser per device, by hand, in code — which is exactly the cost the problem statement complains about. Commercial products like Cribl automate the normalization, but they were built to cut SIEM bills, so they filter and discard by design, most keep data in the cloud, and none of them preserve the original bytes or prove a record is unaltered.

Q13 Twelve other teams picked this statement. Why yours?

TRAP: listing features. Name one constraint you took seriously.

OUR ANSWER

We took the word *lossless* literally. Most teams will normalize logs — that is the obvious reading. We noticed the statement says preserve raw event data *without information loss*, and that this comes from an agency whose logs become evidence. So we store the original bytes before parsing, and we can prove any single record is unaltered without revealing the rest of the log. That is a different system, not a better dashboard.

Q14 Does this already exist? What is genuinely new?

OUR ANSWER

The category exists — seven commercial platforms, Cribl being the leader at over $300M revenue. The individual techniques are all older than us: hash chains, Merkle trees from Certificate Transparency, OCSF. What we did not find anywhere is the combination: store-before-parse, parsers as reviewable data files, a vendor-neutral schema, and per-record cryptographic proofs, in one air-gapped binary. The most detailed market comparison published this year does not mention integrity or chain of custody once — the category has no column for it.

D. About deployment and reality

Q15 What is the smallest version you could deploy next month?

OUR ANSWER

One collector on one machine, receiving syslog from a single building's firewalls on port 5514, writing to local disk and forwarding to the existing SIEM. That is one command and no changes to the devices beyond adding a syslog destination. Everything else — sharding, proofs, the console — is already there and can be used when they are ready.

Q16 This works on your laptop. What breaks in the field?

TRAP: "Nothing, it's production ready."

OUR ANSWER

Three things, and we know each one. **One:** above 10,000 records/second, UDP drops records in the kernel before we see them — we raise the buffer and print what the system granted, but UDP has no backpressure; TCP is future work. **Two:** nine of our 34 parsers have never seen real traffic from that vendor, and real data has broken parsers before. **Three:** the console has no authentication, so it must sit behind a proxy if exposed.

Q17 How does it scale to billions of events a day?

OUR ANSWER

Honestly: one collector does not. We measure 10,000/sec lossless, which is 864 million a day — 86% of a billion. We do not quote a higher number because the next step up drops 1.7% of records. The answer is more collectors, not a faster one, because the store and the chain are single-writer by design and that is what makes the proofs meaningful. Each collector is fully independent; we have tested two together with zero loss and each chain verifying separately.

Q18 What law or policy does this have to comply with?

OUR ANSWER

For evidence, the relevant idea is chain of custody — showing an item is what it claims to be and was not altered. Courts increasingly expect cryptographic verification rather than operator testimony. Our fingerprint plus proof gives that for the data itself. We are honest about the gap: we do not yet record *who read* a record, which a complete custody procedure also requires.

Q19 Which of our existing systems would this talk to?

OUR ANSWER

It sits *in front* of whatever SIEM they already run, so it does not replace anything. Devices point their syslog at us instead of directly at the SIEM, and we forward normalized records onward — we support OpenSearch and Splunk directly, plus Parquet files for a data lake. The original bytes stay with us, which is the part their current setup does not have.

Q20 Who owns this on day one?

OUR ANSWER

The SOC team that already operates the SIEM — they are the ones writing parsers today, so this removes work they currently do. The person who benefits most immediately is the analyst who approves a new parser: what takes a developer hours becomes a review of a text file.

E. About the team and the code

Q21 Open the code for the feature you just showed me.

TRAP: hunting through folders while they watch.

OUR ANSWER

Have the repository already open. Know these four: `crates/ulpf-cli/src/pipeline.rs` (the five stages, 251 lines), `crates/ulpf-ocsf/src/merkle.rs` (proofs), `crates/ulpf-vault/src/format.rs` (the store), `packs/openssh-auth.yaml` (a readable parser).

Q22 How much is from an existing open-source project?

TRAP: saying none of it.

OUR ANSWER

The OCSF schema is Apache-licensed and vendored unmodified — we credit it and did not write it. We use standard Rust libraries for compression, hashing, signatures and the web server. The pipeline, the store format, the Source Pack system, the integrity design and the generator are ours. The Drain clustering idea comes from a published algorithm; the sample-selection method comes from two research papers we cite in the code.

Q23 You have not spoken yet. What did you build?

OUR ANSWER

Every member must be able to answer for their own area *and* the one next to it. Split it before the event: store and integrity; decoders and packs; console and outputs; generator and measurement. Do not let one person answer everything — judges specifically test for this.

Q24 What did you cut, and why?

OUR ANSWER

Three things. Binary formats like NetFlow, because our decoders read text and supporting them needs a second interface, not another parser. TCP and TLS syslog, because UDP covers the demonstrable case and we removed the unused code rather than advertise a capability we did not have. And a login for the console, which we documented as a limitation instead of half-building.

Q25 What did you get wrong and change?

OUR ANSWER

Several, and they are in Part 8. The clearest: we believed our parsers were good because they passed their own tests. Then real data showed parsers scoring 100% on their own tests and 5%, 29% and 37% on real logs — and one at 0% because the vendor's manual was wrong. We changed how we work: measure against real captures before believing anything.

Q26 What would you do with another week?

TRAP: grand vague plans.

OUR ANSWER

Two specific things. Generate real logs for four of our nine unproven parsers — Suricata, Zeek, ModSecurity and pfSense are open source, so we can run the actual software over public network captures and get genuine output. And run a real multi-machine throughput test, which is the one claim we currently support with arithmetic rather than measurement.

Q27 If we gave you three months and a budget, what would you fix first?

OUR ANSWER

Cross-collector witness co-signing — two collectors counter-signing each other's checkpoints. It removes the last single point of trust, it is genuinely distributed without needing a blockchain, and nothing in the commercial market does it. Second would be authentication on the console, because that is our most obvious gap.

Q28 What did you learn that you did not expect?

OUR ANSWER

That passing tests means almost nothing. Every serious bug we found was invisible to a green test run — a parser stealing another's traffic, a generator producing parsers that could never match, full coverage on a log that had become unsearchable. What caught all of them was measuring against real data we did not create.

Q29 How long would it take an operator to learn this?

OUR ANSWER

To use the console and read events: about ten minutes. To approve a machine-drafted parser: roughly an afternoon, because they need to understand what the recognition rules and field mappings mean. To write one from scratch: a day with an existing parser as a template.

Q30 What does it cost to run for a year?

OUR ANSWER

Almost nothing in licensing — it is one binary with no per-gigabyte fee, which is itself the argument, since that fee is why this market exists. The real cost is storage: at a billion events a day, roughly 150 GB of text daily, compressed better than 4:1, so about 35–40 GB a day of retained originals. Five collectors on commodity servers. The largest line item is disk, not compute.

Q31 Would you keep working on this?

OUR ANSWER

Only say yes if it is true. If it is, be specific about which part — the integrity work is the piece with genuine research value, and the gap we found in the commercial market is real.

PART 11

Our honest weaknesses

Know these before a judge finds them. Volunteering a weakness with a mitigation reads as competence; being caught hiding one does the opposite.

Weakness	What we say
Nine of 34 parsers have no real-data proof	Cisco ASA, FortiGate, PAN-OS, Check Point, Juniper, Suricata, ModSecurity, generic CEF and pfSense. They pass tests written from vendor manuals — and we have seen a manual be wrong. We list them openly. Four are open-source software we can generate real logs from.
One collector does not reach a billion a day	10,000/sec is 86% of the target. Sharding is built and tested on one machine; a genuine multi-machine measurement has not been run.
The console has no login	It blocks cross-site and rebound requests, but that is browser safety, not authentication. Needs a proxy in front for any wider exposure.
No binary formats	NetFlow and Windows Event Log are not supported. Our decoders read text; these need a second interface. NetFlow is genuinely in scope, so this is a real gap.
Outputs use plain HTTP	Forwarding to a SIEM assumes a trusted local endpoint; TLS is the deployment's job.
The AI assistant is weak	It runs a small model so it works on any hardware, and it shows — it hedges and occasionally invents details. That is exactly why the rules-based generator exists and why no model ever runs in the processing path.
Coverage is 99.92%, not 100%	The remainder is listed by name in our dataset notes. Unparsed records are still stored, still sealed, and still searchable through salvage.

THE FRAMING THAT WORKS

"We would rather show you a number we can defend than a rounder one we cannot." Every weakness above is already written in our README. A judge who goes looking will find that we got there first — which is worth more than a perfect-sounding claim.

PART 12

One-page cheat sheet

The last thing to read before you walk in.

The pitch, in thirty seconds

> "Security teams get logs from dozens of devices in incompatible formats, and normal tools throw away the original while converting them — so when an investigation needs the evidence, it is gone. ULPF stores the exact original bytes *before* parsing, converts everything to one open standard, and keeps cryptographic proof that no record has been altered. Adding a new device is a text file, not code. It runs with no internet connection at all."

Numbers to know cold

Real records tested	**838,779**	Coverage	**99.9219%**
Source Packs	**34**	Tests passing	**64 / 64**
Speed, lossless	**10,000/sec**	Per day	**864 million**
Proof size	**352 bytes**	Container	**65.4 MB**

The five stages

Vault → Detect → Read → Convert → Seal. The vault comes first, and that ordering is the whole design.

Three things that make us different

1. **Store before parsing.** Nothing after that line can lose the original.
2. **Parsers are data, not code.** A text file, reviewed and hot-loaded.
3. **Prove one record without showing the rest.** 352 bytes, verified by someone holding no other part of the log.

If you only rehearse three answers

- **Why Rust?** Speed, memory safety on attacker-controlled input, single file for air-gapped machines.
- **Does it already exist?** The category does; the combination does not. The biggest 2026 market comparison has no column for integrity.

- **What breaks in the field?** UDP loss above 10,000/sec; nine parsers without real-data proof; no login on the console.

> **LAST THING**
>
> Lead with the running system, not the slides. Sixty seconds: live events, then the tamper test. Then let them ask.